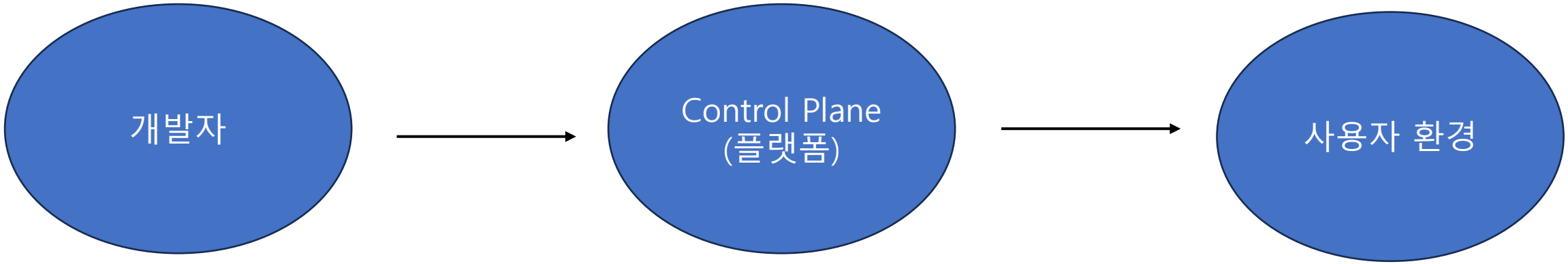
Self-Service Cloud Platform

이진서

Self-Service Cloud Platform이란

- 개발자가 인프라를 몰라도 앱 요구사항에 답하면,
- 플랫폼이 알맞은 AWS 구조를 골라 만들고 검증까지 마친 환경을 만들어 주는 플랫폼이다



Platform 흐름

1. **사용자가 애플리케이션 요구사항 입력**

- 트래픽,응답 시간, 가용성, 저장소, 예산 등

↓

2. **플랫폼이 AWS 아키텍처 추천**

- Static / Serverless / Container

↓

3. **승인 -> Terraform 배포 -> 검증**

요구사항

- 1. 앱 형태: 화면만 / 서버 앱 / 화면 + API
- 2. 특수 기능: 실시간 연결 · 큰 파일 · 서버 메모리 상태 (서버 O)
- 3. 가장 긴 요청 시간 (서버 O)
- 4. 하루 사용자 규모
- 5. 트래픽 패턴
- 6. 데이터 저장 필요 여부 (DB 종류)
- 7. 일시 중단 허용 여부
- 8. 첫 요청 지연(Cold Start) 허용 여부
- 9. 공개 범위
- 10. 용도(개발·테스트 / 운영)와 월 예산 상한

베이스 라인 애플리케이션

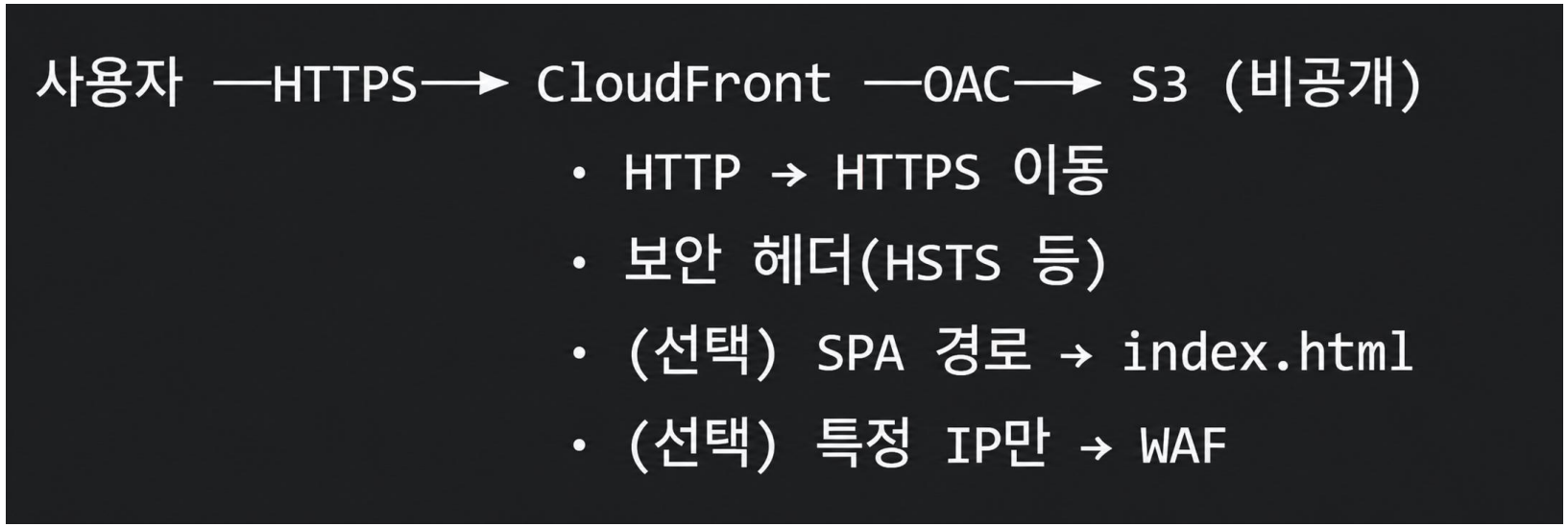
Static / Serverless / Container

Static

- 앱 형태 = **화면만 있는 사이트** 이면 Static.

(HTML·CSS·JS 파일, React·Vue 빌드 결과 포함)

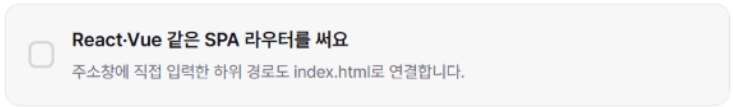
1. 질문 답변 → 2. 빌드 결과 zip 업로드 → 3. 승인 → 4. HTTPS 주소 받기



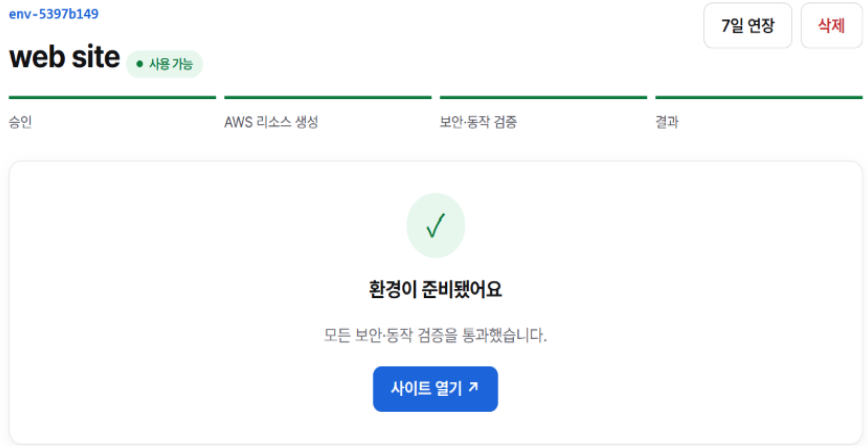
Demo

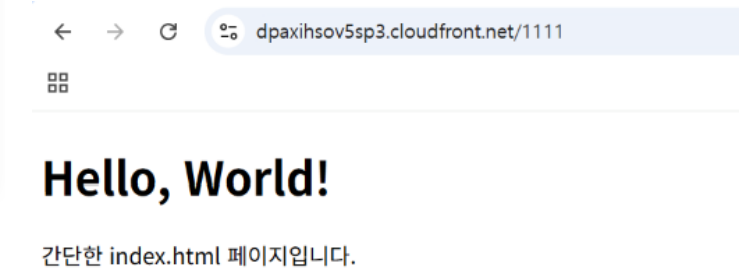

1 / 8
어떤 앱을 올리시나요?
만든 결과물의 형태를 골라 주세요.
화면만 있는 사이트
HTML·CSS·JS 파일 (React·Vue 빌드 결과 포함)
서버 앱
API 서버나 서버에서 화면을 만드는 앱
화면 + API
화면 파일과 API 서버를 함께 운영


사이트 파일 올리기
빌드 결과 폴더(index.html 포함)를 zip으로 묶어 올려 주세요. 50MB 이하.
여기에 zip 파일을 끌어 놓거나 눌러서 고르세요


React·Vue 같은 SPA 라우터를 써요
주소창에 직접 입력한 하위 경로도 index.html로 연결합니다.


추천 결과
정적 사이트 구성을 추천해요
web site · 개발·테스트용
구성
사용자
브라우저
CloudFront
HTTPS · 보안 헤더
S3 (비공개)
화면 파일
이렇게 고른 이유
화면 파일은 서버 없이 CDN(CloudFront)으로 바로 제공합니다. SEL_STATIC_FRONTEND
조건을 모두 만족하는 후보가 이것 하나입니다. SEL_ONLY_CANDIDATE
저장할 데이터가 없어서 데이터베이스를 만들지 않습니다. SEL_DB_NONE
CloudFront·S3은(는) 추가 비용 없이 여러 가용 영역에 걸쳐 동작합니다. SEL_AZ_INHERENT
필수 보안 기준 11개 항목이 모두 적용됩니다. SEC_BASELINE_APPLIED
적용되는 설정
개발·테스트용 환경 정책을 적용합니다(로그 7일 보관, 자동 만료). POL_TIER_DEV
7일 뒤 자동으로 삭제됩니다(연장 가능). POL_TTL_DEV


env-5397b149
web site
사용 가능
7일 연장
삭제
승인
AWS 리소스 생성
보안·동작 검증
결과
환경이 준비됐어요
모든 보안·동작 검증을 통과했습니다.
사이트 열기


dpaxihsov5sp3.cloudfront.net/1111
Hello, World!
간단한 index.html 페이지입니다.

Serverless

- 앱 형태 = 서버 앱 + 짧은 요청

(30초 안에 끝나는 API, 실시간 연결·큰 파일·서버 메모리 상태 없음)

1. 질문 답변 → 2. image 주소·포트 입력 → 3. 승인 → 4. HTTPS 주소 받기

[빌드] 공개 image → 플랫폼이 Lambda Web Adapter를 없은 image 생성 → ECR (비공개)

[요청] 사용자 —HTTPS→ API Gateway → Lambda

- 요청이 올 때만 실행 (없으면 버용 거의 0)
- 접속 로그 + 보안 헤더
- Lambda로 바로 들어오는 주소 없음
- 일반 웹 서버 image 그대로 (코드 수정 없음)

Demo

1 / 10

어떤 앱을 올리시나요?

만든 결과물의 형태를 골라 주세요.

- ○ **화면만 있는 사이트**
  HTML·CSS·JS 파일 (React·Vue 빌드 결과 포함)
- ◉ **서버 앱**
  API 서버나 서버에서 화면을 만드는 앱
- ○ **화면 + API**
  화면 파일과 API 서버를 함께 운영

▶ 예시 보기

앱 컨테이너 이미지

공개 저장소(docker.io, public.ecr.aws, ghcr.io, quay.io)에 있는 HTTP 서버 이미지를 적어 주세요. 앱은 PORT 환경변수(8080)로 요청을 받아야 해요. 플랫폼이 원본은 그대로 두고, Lambda에서 실행할 수 있게 Lambda Web Adapter를 얹은 이미지를 따로 만들어요.

이미지 주소 (태그 또는 digest 포함)

`ghcr.io/mccutchen/go-httpbin:v2.16.1`

상태 확인 경로

`/`

이 경로가 200으로 응답하면 앱이 준비된 것으로 봐요.

환경변수 (한 줄에 하나, KEY=VALUE)

`WHOAMI_PORT_NUMBER=8080`

비밀번호·토큰 같은 비밀값은 넣지 마세요. 아직 비밀값 보관(Secrets Manager)을 지원하지 않아서, 비밀값으로 보이면 만들기 전에 막아요.

t0344c0t3j.execute-api.ap-northeast-2.amazonaws.com

업데이트를 위해 다시 시작

go-httpbin(1)

유서 깊은 httpbin.org HTTP 요청 및 응답 테스트 서비스를 Go 언어로 포팅한 것입니다.

종점

/이 페이지.
/absolute-redirect/:n302개의 절대 리디렉션이 *n* 번 발생합니다.
/anything/:anythingrequest에 전달된 모든 값을 반환합니다.
/base64/:value?content-type=ct선택적으로 Content-Type을 지정하여 Base64로 인코딩된 문자열을 디코딩합니다.
/base64/decode/:value?content-type=ctBase64로 인코딩된 문자열을 디코딩하기 위한 명시적 URL입니다.
/base64/encode/:value문자열을 URL에 안전한 Base64로 인코딩합니다.
/basic-auth/:user/:passwordHTTPBasic 인증에 대한 문제점.
/bearer베어러 토큰 헤더를 확인합니다. 설정되어 있지 않으면 401 오류를 반환합니다.
~~/brotli~~브로틀리 인코딩된 데이터를 반환합니다. *아직 구현되지 않았습니다!*
/bytes/:n*n* 바이트의 이진 데이터를 무작위로 생성하며 , 선택적으로 *s*/*d* 정수 매개변수를 받을 수 있습니다.
/cacheIf-Modified-Since 또는 If-None-Match 헤더가 제공되지 않은 경우 200을 반환하고, 제공된 경우에는 304를 반환합니다.
/cache/:n*n* 초 동안 Cache-Control 헤더를 설정합니다 .
/cookies쿠키 데이터를 반환합니다.
/cookies/delete?name하나 이상의 일반 쿠키를 삭제합니다.
/cookies/set?name=value하나 이상의 간단한 쿠키를 설정합니다.
/deflatedeflate 인코딩된 데이터를 반환합니다.
/delay/:n응답 지연 시간은 *min(n, 10)* 초입니다.
/delete요청 데이터를 반환합니다. DELETE요청만 허용합니다.
/denyrobots.txt 파일에 의해 차단되었습니다.

2 / 10

서버 앱에 이런 기능이 있나요?

해당하는 것을 모두 골라 주세요. 없으면 그냥 넘어가도 됩니다.

- ☐ **실시간 연결**
  채팅, 실시간 알림처럼 연결을 계속 유지
- ☐ **큰 파일 업로드·다운로드**
  한 번에 6MB가 넘는 요청이나 응답
- ☐ **서버 메모리·로컬 디스크에 상태 보관**
  로그인 상태나 업로드 파일을 서버 안에 두고 다음 요청에서 다시 씀

▼ 예시 보기

- **"실시간 연결"** — 채팅, 실시간 알림, 공동 편집
- **"큰 파일 업로드·다운로드"** — 동영상·대용량 파일 업로드
- **"서버 메모리·로컬 디스크에 상태 보관"** — 로그인 정보를 서버 메모리에 보관

진행 기록

26. 10. 11. 오전 1:00	● 사용 가능	모든 검증을 통과했습니다. 환경을 사용할 수 있습니다.
26. 10. 11. 오전 1:00	● 검증 중	리소스 생성이 끝나 보안·동작 검증을 시작합니다.
26. 10. 11. 오전 1:00		리소스를 생성했습니다: https://t0344c0t3j.execute-api.ap-northeast-2.amazonaws.com
26. 10. 11. 오전 1:00		PLAN 검증(2/2 · 최종 구성, 이미지 digest 고정)을 통과했습니다.
26. 10. 11. 오전 1:00		Lambda용 이미지를 만들었습니다 (sha256:94883732c9b8...). 원본 이미지는 바꾸지 않았습니다.
26. 10. 11. 오전 12:59		이미지 저장소를 만들고 Lambda용 이미지를 준비합니다.
26. 10. 11. 오전 12:59		PLAN 검증(1/2 · 전체 구성, 이미지 확정 전)을 통과했습니다.
26. 10. 11. 오전 12:59	● 생성 중	Terraform plan과 보안 검사를 실행합니다.
26. 10. 11. 오전 12:59	● 생성 중	승인되어 AWS 리소스 생성을 시작합니다.
26. 10. 11. 오전 12:59	● 초안	추천 결과와 함께 환경 초안을 만들었습니다.

Self-Service Cloud

+ 새 환경 만들기 | 전체 환경 | 로그아웃

env-c0a4184c

serverless ● 사용 가능

7일 연장 | 삭제

승인	AWS 리소스 생성
보안·동작 검증	결과

✓

환경이 준비됐어요

모든 보안·동작 검증을 통과했습니다.

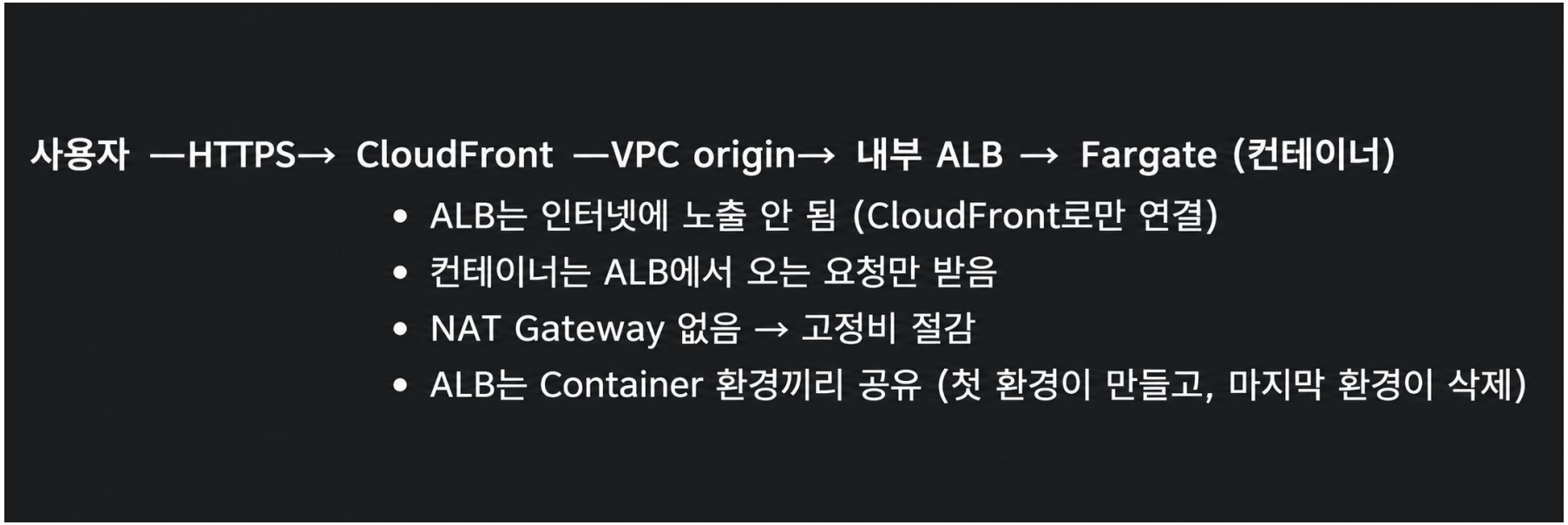
사이트 열기 ↗

Container

- 앱 형태 = 서버 앱 + Lambda로는 안 되는 조건

(실시간 연결 · 30초~2분 요청 · 서버 메모리 상태 · 큰 파일 · 특정 IP만)

1. 질문 답변 → 2. image 주소·health 경로 입력 → 3. 승인 → 4. HTTPS 주소 받기



Demo

1 / 10

어떤 앱을 올리시나요?

만든 결과물의 형태를 골라 주세요.

- ○ **화면만 있는 사이트**
  HTML·CSS·JS 파일 (React·Vue 빌드 결과 포함)
- ◉ **서버 앱**
  API 서버나 서버에서 화면을 만드는 앱
- ○ **화면 + API**
  화면 파일과 API 서버를 함께 운영

2 / 10

서버 앱에 이런 기능이 있나요?

해당하는 것을 모두 골라 주세요. 없으면 그냥 넘어가도 됩니다.

- ☑ **실시간 연결**
  채팅, 실시간 알림처럼 연결을 계속 유지
- ☐ **큰 파일 업로드·다운로드**
  한 번에 6MB가 넘는 요청이나 응답
- ☐ **서버 메모리·로컬 디스크에 상태 보관**
  로그인 상태나 업로드 파일을 서버 안에 두고 다음 요청에서 다시 씀

▶ 예시 보기

3 / 10

가장 오래 걸리는 요청은 얼마나 걸리나요?

버튼을 누르고 결과가 나올 때까지 걸리는 시간 중 가장 긴 것 하나를 골라 주세요.

- ○ **대부분 몇 초 안에 끝나요**
- ◉ **30초 정도 걸리는 작업도 있어요**
- ○ **30초~1분 걸리는 작업도 있어요**
- ○ **1~2분 걸리는 작업도 있어요**
- ○ **2분 넘게 걸리는 작업도 있어요**

▶ 예시 보기

9 / 10

누가 접속하나요?

- ○ **누구나**
  인터넷에 공개
- ○ **로그인한 사용자만**
  로그인은 앱이 처리
- ◉ **특정 IP에서만**
  회사·학교 네트워크 등

허용할 IP 주소 범위

예: 203.0.113.0/24, 198.51.100.7/32

회사·학교 네트워크 담당자에게 받은 주소를 쉼표로 구분해 적어 주세요. 한 대만 허용하려면 끝에 /32를 붙입니다.

▶ 예시 보기

앱 컨테이너 이미지

공개 저장소(docker.io, public.ecr.aws, ghcr.io, quay.io)에 있는 HTTP 서버 이미지를 적어 주세요. 앱은 PORT 환경변수(8080)로 요청을 받아야 해요. 플랫폼이 원본 이미지를 그대로 환경 전용 저장소로 복사해 ECS Fargate에서 실행해요.

컨테이너는 요청이 없어도 항상 실행돼서 고정비가 있어요 (컨테이너 월 약 $7). 공유 부하분산기(내부 ALB)는 컨테이너 환경들이 나눠 쓰는 것으로 계산했어요. 지금 컨테이너 환경이 하나도 없다면 실제로는 월 약 $16이 더 들고, 마지막 컨테이너 환경을 지우면 함께 삭제돼요. 처음 만들 때는 공유 인프라 준비로 20분 정도 걸릴 수 있어요.

이미지 주소 (태그 또는 digest 포함)

```
ghcr.io/mccutchen/go-httpbin:v2.16.1
```

상태 확인 경로

```
/status/200
```

이 경로가 200으로 응답하면 앱이 준비된 것으로 봐요.

환경변수 (한 줄에 하나, KEY=VALUE)

```
MAX_DURATION=50s
USE_REAL_HOSTNAME=true
```

비밀번호·토큰 같은 비밀값은 넣지 마세요. 아직 비밀값 보관(Secrets Manager)을 지원하지 않아서, 비밀값으로 보이면 만들기 전에 막아요.

들어주셔서 감사합니다